

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 3**



Build a Scrollable List

Oleh:

Rika Fauliana Rahmi NIM. 2410817120017

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 3

Laporan Praktikum Pemrograman Mobile Modul 3: Build a Scrollable List ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Rika Fauliana Rahmi
NIM : 2410817120017

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,
M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1	6
A. Source Code.....	9
B. Output Program	34
C. Pembahasan	38
D. Tautan Git	48

DAFTAR GAMBAR

Gambar 1. Contoh UI List	7
Gambar 2. Contoh UI Detail.....	8
Gambar 3. Contoh Penerapan LazyRow/Carousel	8
Gambar 4. Screenshot Hasil Jawaban Soal 1 XML.....	34
Gambar 5. Screenshot Hasil Jawaban Soal 1 XML.....	35
Gambar 6. Screenshot Hasil Jawaban Soal 1 XML.....	35
Gambar 7. Screenshot Hasil Jawaban Soal 1 XML.....	36
Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose	36
Gambar 9. Screenshot Hasil Jawaban Soal 1 Compose	37
Gambar 10. Screenshot Hasil Jawaban Soal 1 Compose	37
Gambar 11. Screenshot Hasil Jawaban Soal 1 Compose	38

DAFTAR TABEL

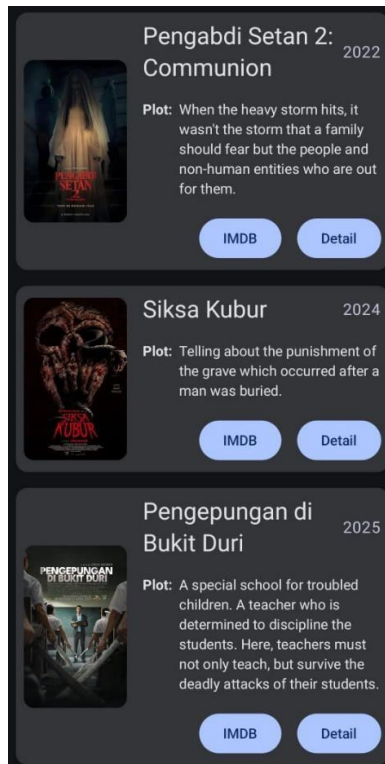
Tabel 1. Source Code MainActivity.kt XML Soal 1	9
Tabel 2. Source Code Lego.kt XML Soal 1	9
Tabel 3. Source Code LegoViewModel.kt XML Soal 1	10
Tabel 4. Source Code LegoAdapter.kt XML Soal 1	11
Tabel 5. Source Code ListFragment.kt XML Soal 1	12
Tabel 6. Source Code DetailFragment.kt XML Soal 1	14
Tabel 7. Source Code LanguageFragment.kt XML Soal 1	15
Tabel 8. Source Code HighkightAdapter.kt XML Soal 1	16
Tabel 9. Source Code activity_main.xml XML Soal 1	17
Tabel 10. Source Code item_lego.xml XML Soal 1	17
Tabel 11. Source Code item_highlight.xml XML Soal 1	20
Tabel 12. Source Code fragment_list.xml XML Soal 1	21
Tabel 13. Source Code fragment_detail.xml XML Soal 1	23
Tabel 14. Source Code fragment_language.xml XML Soal 1	23
Tabel 15. Source Code nav_graph.xml XML Soal 1	24
Tabel 16. Source Code strings.xml (en) XML Soal 1	25
Tabel 17. Source Code strings.xml (in) XML Soal 1	25
Tabel 18. Source Code MainActivity.kt Compose Soal 1	26
Tabel 19. Source Code Lego.kt Compose Soal 1	27
Tabel 20. Source Code LegoItem.kt Compose Soal 1	28
Tabel 21. Source Code AppScreens.kt Compose Soal 1	30
Tabel 22. Source Code strings.xml (in)Compose Soal 1	33
Tabel 23. Source Code strings.xml (en)Compose Soal 1	34

SOAL 1

1. Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:
 1. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
 2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
 3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
 4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - b. Button kedua menggunakan Navigation component untuk membuka laman detail item
 5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
 6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
 7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
 8. Aplikasi berbasis XML harus menggunakan ViewBinding
 9. Aplikasi Mendukung Multi-language (bahasa indonesia dan inggris)
 10. Terdapat 1 halaman/screen untuk melakukan pergantian Bahasa
 11. Gunakan Icon dari library 'material' (material,material2,material3) untuk membuat tombol yang mengarahkan ke halaman untuk melakukan pergantian bahasa, dan letakkan di bagian header/heading homescreen utama
 12. Diatas Scrollable list, terdapat sebuah "Item Highlight" yang menggunakan Carousel atau LazyRow dengan ketentuan:
 - Item bisa bergeser secara finite atau infinite loop sebanyak jumlah item yang ada pada scrollable-list
2. Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

- 3 Pada desain arsitektur kotlin, terdapat beberapa arsitektur dengan salah satunya adalah CLEAN ARCHITECTURE, menurut pendapat kalian, mengapa kita harus menerapkan arsitektur ini ketika membuat sebuah project?

UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Diusahakan agar desain UI item list menyerupai UI berikut:

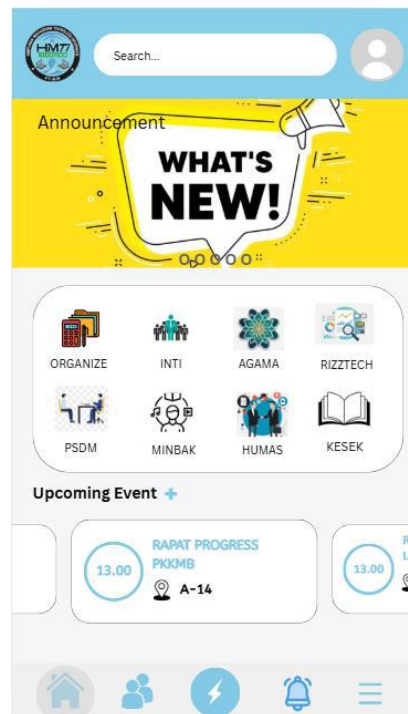


Gambar 1. Contoh UI List

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 2. Contoh UI Detail



Gambar 3. Contoh Penerapan LazyRow/Carousel

Item yang bergeser bisa secara finite (mentok kiri kanan) atau secara infinite loop (misal dari item terakhir '5' kembali ke item awal '1')

A. Source Code

1. XML

MainActivity.kt XML

Tabel 1. Source Code MainActivity.kt XML Soal 1

```
1 package com.example.modul3xml
2
3 import android.os.Bundle
4 import androidx.appcompat.app.AppCompatActivity
5 import
  com.example.modul3xml.databinding.ActivityMainBinding
6
7 class MainActivity : AppCompatActivity() {
8     private lateinit var binding: ActivityMainBinding
9
10    override fun onCreate(savedInstanceState: Bundle?) {
11        super.onCreate(savedInstanceState)
12
13        binding =
  ActivityMainBinding.inflate(layoutInflater)
14        setContentView(binding.root)
15    }
16 }
```

Lego.kt XML

Tabel 2. Source Code Lego.kt XML Soal 1

```
1 package com.example.modul3xml
2
3 import android.os.Parcel
4 import android.os.Parcelable
5
6 data class Lego(
7     val id: Int,
8     val title: String,
9     val year: String,
10    val theme: String,
11    val pieces: String,
12    val description: String,
13    val imageRes: Int,
14    val webUrl: String
15 ) : Parcelable {
16
17    constructor(parcel: Parcel) : this(
18        parcel.readInt(),
19        parcel.readString() ?: "",
```

```

20         parcel.readString() ?: "",
21         parcel.readString() ?: "",
22         parcel.readString() ?: "",
23         parcel.readString() ?: "",
24         parcel.readInt(),
25         parcel.readString() ?: ""
26     )
27
28     override fun writeToParcel(parcel: Parcel, flags:
Int) {
29         parcel.writeInt(id)
30         parcel.writeString(title)
31         parcel.writeString(year)
32         parcel.writeString(theme)
33         parcel.writeString(pieces)
34         parcel.writeString(description)
35         parcel.writeInt(imageRes)
36         parcel.writeString(webElement)
37     }
38
39     override fun describeContents(): Int {
40         return 0
41     }
42
43     companion object CREATOR : Parcelable.Creator<Lego>
{
44         override fun createFromParcel(parcel: Parcel):
Lego {
45             return Lego(parcel)
46         }
47
48         override fun newArray(size: Int): Array<Lego?> {
49             return arrayOfNulls(size)
50         }
51     }
52 }

```

LegoViewModel.kt XML

Tabel 3. Source Code LegoViewModel.kt XML Soal 1

```

1 package com.example.modul3xml
2
3 import androidx.lifecycle.ViewModel
4
5 class LegoViewModel : ViewModel() {
6     val legoList = listOf(

```

7	Lego(1, "Millennium Falcon", "2017", "Star Wars", "7541 pcs", "Pesawat tempur legendaris Han Solo.", R.drawable.lego1, "https://www.lego.com"),
8	Lego(2, "Hogwarts Castle", "2018", "Harry Potter", "6020 pcs", "Kastil sekolah sihir Hogwarts.", R.drawable.lego2, "https://www.lego.com"),
9	Lego(3, "Taj Mahal", "2021", "Architecture", "2022 pcs", "Replika bangunan Taj Mahal.", R.drawable.lego3, "https://www.lego.com"),
10	Lego(4, "Porsche 911", "2021", "Creator", "1458 pcs", "Mobil klasik Porsche 911.", R.drawable.lego4, "https://www.lego.com"),
11	Lego(5, "Central Perk", "2019", "Ideas", "1070 pcs", "Kafe ikonik dari serial TV Friends.", R.drawable.lego5, "https://www.lego.com")
12	)
13	}

LegoAdapter.kt XML

Tabel 4. Source Code LegoAdapter.kt XML Soal 1

1	package com.example.modul3xml
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import android.view.LayoutInflater
7	import android.view.ViewGroup
8	import androidx.navigation.findNavController
9	import androidx.recyclerview.widget.RecyclerView
10	import com.example.modul3xml.databinding.ItemLegoBinding
11	
12	class LegoAdapter(private val listData: List<Lego>) : RecyclerView.Adapter<LegoAdapter.LegoViewHolder>() {
13	
14	class LegoViewHolder(val binding: ItemLegoBinding) : RecyclerView.ViewHolder(binding.root)
15	
16	override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): LegoViewHolder {
17	val view = ItemLegoBinding.inflate(LayoutInflater.from(parent.context), parent, false)
18	return LegoViewHolder(view)
19	}
20	

21	override fun onBindViewHolder(holder: LegoViewHolder, position: Int) {
22	val item = listData[position]
23	
24	holder.binding.tvTitle.text = item.title
25	holder.binding.tvYear.text = item.year
26	holder.binding.tvTheme.text = item.theme
27	holder.binding.tvDescription.text =
	item.description
28	
	holder.binding.ivLego.setImageResource(item.imageRes)
29	
30	holder.binding.btnWeb.setOnClickListener {
31	val link = Uri.parse(item.webUrl)
32	val intentWeb = Intent(Intent.ACTION_VIEW,
	link)
33	
	holder.itemView.context.startActivity(intentWeb)
34	}
35	
36	holder.binding.btnDetail.setOnClickListener {
37	val bundle = Bundle()
38	bundle.putParcelable("data_lego", item)
39	
	holder.itemView.findNavController().navigate(R.id.action_list_to_detail, bundle)
40	}
41	}
42	
43	override fun getItemCount(): Int = listData.size
44	}

ListFragment.kt XML

Tabel 5. Source Code ListFragment.kt XML Soal 1

1	package com.example.modul3xml
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.fragment.app.Fragment
8	import androidx.lifecycle.ViewModelProvider
9	import androidx.navigation.fragment.findNavController
10	import androidx.recyclerview.widget.LinearLayoutManager
11	import
	com.example.modul3xml.databinding.FragmentListBinding

```

12
13 class ListFragment : Fragment() {
14
15     private var _binding: FragmentListBinding? = null
16     private val binding get() = _binding!!
17
18     override fun onCreateView(
19         inflater: LayoutInflater, container: ViewGroup?,
20         savedInstanceState: Bundle?
21     ): View {
22         _binding = FragmentListBinding.inflate(inflater,
23         container, false)
24         return binding.root
25     }
26
27     override fun onViewCreated(view: View,
28         savedInstanceState: Bundle?) {
29         super.onViewCreated(view, savedInstanceState)
30
31         val viewModel =
32         ViewModelProvider(this).get(LegoViewModel::class.java)
33
34         val layoutManagerHorizontal =
35         LinearLayoutManager(requireContext(),
36         LinearLayoutManager.HORIZONTAL, false)
37         binding.rvHighlight.layoutManager =
38         layoutManagerHorizontal
39
40         val adapterHighlight =
41         HighlightAdapter(viewModel.legoList)
42         binding.rvHighlight.adapter = adapterHighlight
43
44         binding.rvLego.layoutManager =
45         LinearLayoutManager(requireContext())
46         val adapterLego =
47         LegoAdapter(viewModel.legoList)
48         binding.rvLego.adapter = adapterLego
49
50         binding.btnLanguage.setOnClickListener {
51             findNavController().navigate(R.id.action_listFragment_t
52             o_languageFragment)
53         }
54     }
55
56     override fun onDestroyView() {
57         super.onDestroyView()
58     }
59 }

```

48	<code>_binding = null</code>
49	<code>}</code>
50	<code>}</code>

DetailFragment.kt XML

Tabel 6. Source Code DetailFragment.kt XML Soal 1

1	<code>package com.example.modul3xml</code>
2	
3	<code>import android.os.Bundle</code>
4	<code>import android.view.LayoutInflater</code>
5	<code>import android.view.View</code>
6	<code>import android.view.ViewGroup</code>
7	<code>import androidx.fragment.app.Fragment</code>
8	<code>import</code>
9	<code>com.example.modul3xml.databinding.FragmentDetailBinding</code>
10	<code>class DetailFragment : Fragment() {</code>
11	
12	<code>private var _binding: FragmentDetailBinding? = null</code>
13	<code>private val binding get() = _binding!!</code>
14	
15	<code>override fun onCreateView(</code>
16	<code>inflater: LayoutInflater, container: ViewGroup?,</code>
17	<code>savedInstanceState: Bundle?</code>
18	<code>): View {</code>
19	<code> _binding</code>
20	<code>FragmentDetailBinding.inflate(inflater, container,</code>
21	<code>false)</code>
22	<code> return binding.root</code>
23	<code> }</code>
24	<code>override fun onViewCreated(view: View,</code>
25	<code>savedInstanceState: Bundle?) {</code>
26	<code> super.onViewCreated(view, savedInstanceState)</code>
27	<code> val legoDiterima</code>
28	<code>arguments?.getParcelable<Lego>("data_lego")</code>
29	<code> if (legoDiterima != null) {</code>
30	<code> binding.ivDetail.setImageResource(legoDiterima.imageRes)</code>
31	<code> binding.tvTitleDetail.text</code>
32	<code>legoDiterima.title</code>

33	val teksDetail = "Tahun Rilis:
	\${legoDiterima.year}\n" +
34	"Tema: \${legoDiterima.theme}\n" +
35	"Jumlah Kepingan:
	\${legoDiterima.pieces}\n\n" +
36	"Deskripsi:\n\${legoDiterima.description}"
37	
38	binding.tvDescDetail.text = teksDetail
39	}
40	}
41	
42	override fun onDestroyView() {
43	super.onDestroyView()
44	_binding = null
45	}
46	}

LanguageFragment.kt XML

Tabel 7. Source Code LanguageFragment.kt XML Soal 1

1	package com.example.modul3xml
2	
3	import android.content.res.Configuration
4	import android.os.Bundle
5	import android.view.View
6	import androidx.fragment.app.Fragment
7	import
	com.example.modul3xml.databinding.FragmentLanguageBindi
	ng
8	import java.util.Locale
9	
10	class LanguageFragment :
	Fragment(R.layout.fragment_language) {
11	private var _binding: FragmentLanguageBinding? = null
12	private val binding get() = _binding!!
13	
14	override fun onCreateView(view: View,
	savedInstanceState: Bundle?) {
15	super.onCreateView(view, savedInstanceState)
16	_binding = FragmentLanguageBinding.bind(view)
17	
18	binding.btnIndo.setOnClickListener {
	setLocale("in") }
19	
20	binding.btnEnglish.setOnClickListener {
	setLocale("en") }

21	}
22	
23	private fun setLocale(langCode: String) {
24	val locale = Locale(langCode)
25	Locale.setDefault(locale)
26	val config = Configuration()
27	config.setLocale(locale)
28	
29	requireActivity().resources.updateConfiguration(config,
	requireActivity().resources.displayMetrics)
30	
31	requireActivity().recreate()
32	}
33	
34	override fun onDestroyView() {
35	super.onDestroyView()
36	_binding = null
37	}
38	}

HighlightAdapter.kt XML

Tabel 8. Source Code HighkightAdapter.kt XML Soal 1

1	package com.example.modul3xml
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.RecyclerView
6	import
	com.example.modul3xml.databinding.ItemHighlightBinding
7	
8	class HighlightAdapter(private val list: List<Lego>) :
	RecyclerView.Adapter<HighlightAdapter.ViewHolder>() {
9	class ViewHolder(val binding: ItemHighlightBinding)
	: RecyclerView.ViewHolder(binding.root)
10	
11	override fun onCreateViewHolder(parent: ViewGroup,
	viewType: Int): ViewHolder {
12	val binding =
	ItemHighlightBinding.inflate(LayoutInflater.from(parent
	.context), parent, false)
13	return ViewHolder(binding)
14	}
15	
16	override fun onBindViewHolder(holder: ViewHolder,
	position: Int) {

17	holder.binding.ivHighlight.setImageResource(list[position].imageRes)
18	}
19	
20	override fun getItemCount(): Int = list.size
21	}

activity_main.xml

Tabel 9. Source Code activity_main.xml XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.fragment.app.FragmentContainerView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:id="@+id/nav_host_fragment"
6	android:name="androidx.navigation.fragment.NavHostFragment"
7	android:layout_width="match_parent"
8	android:layout_height="match_parent"
9	app:defaultNavHost="true"
10	app:navGraph="@navigation/nav_graph" />

item_lego.xml

Tabel 10. Source Code item_lego.xml XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content"
7	android:layout_margin="8dp"
8	app:cardCornerRadius="16dp"
9	app:cardElevation="4dp"
10	app:cardBackgroundColor="#2C2C2E">
11	
12	<LinearLayout
13	android:layout_width="match_parent"
14	android:layout_height="wrap_content"
15	android:orientation="horizontal"
16	android:padding="12dp">

```

17
18         <androidx.cardview.widget.CardView
19             android:layout_width="100dp"
20             android:layout_height="140dp"
21             app:cardCornerRadius="12dp"
22             app:cardElevation="0dp">
23
24             <ImageView
25                 android:id="@+id/ivLego"
26                 android:layout_width="match_parent"
27                 android:layout_height="match_parent"
28                 android:scaleType="centerCrop" />
29         </androidx.cardview.widget.CardView>
30
31         <LinearLayout
32             android:layout_width="match_parent"
33             android:layout_height="wrap_content"
34             android:orientation="vertical"
35             android:layout_marginStart="16dp">
36
37             <RelativeLayout
38                 android:layout_width="match_parent"
39                 android:layout_height="wrap_content">
40
41                 <TextView
42                     android:id="@+id/tvTitle"
43                     android:layout_width="wrap_content"
44                     android:layout_height="wrap_content"
45                     android:layout_toStartOf="@id/tvYear"
46                     android:layout_alignParentStart="true"
47                     android:textColor="@android:color/white"
48                     android:textSize="18sp"
49                     android:textStyle="bold"
50                     android:maxLines="2"
51                     android:ellipsize="end"/>
52
53                 <TextView
54                     android:id="@+id/tvYear"
55                     android:layout_width="wrap_content"
56                     android:layout_height="wrap_content"
57                     android:layout_alignParentEnd="true"

```

```

58         android:textColor="#BCBCBC"
59         android:textSize="14sp" />
60     </RelativeLayout>
61
62     <LinearLayout
63         android:layout_width="match_parent"
64         android:layout_height="wrap_content"
65         android:orientation="horizontal"
66         android:layout_marginTop="8dp">
67
68         <TextView
69             android:layout_width="wrap_content"
70             android:layout_height="wrap_content"
71             android:text="@string/label_theme"
72             android:textColor="#BCBCBC"
73             android:textStyle="bold"
74             android:textSize="13sp" />
75
76         <TextView
77             android:id="@+id/tvTheme"
78             android:layout_width="wrap_content"
79             android:layout_height="wrap_content"
80             android:layout_marginStart="4dp"
81             android:textColor="#BCBCBC"
82             android:textSize="13sp" />
83     </LinearLayout>
84
85     <TextView
86         android:id="@+id/tvDescription"
87         android:layout_width="match_parent"
88         android:layout_height="wrap_content"
89         android:layout_marginTop="4dp"
90         android:textColor="#8E8E93"
91         android:textSize="12sp"
92         android:maxLines="2"
93         android:ellipsize="end" />
94
95     <LinearLayout
96         android:layout_width="match_parent"
97         android:layout_height="wrap_content"
98         android:gravity="end"
99         android:layout_marginTop="8dp">
100
101         <Button
102             android:id="@+id/btnWeb"

```

103	style="@style/Widget.MaterialComponents.Button.TextButton"
104	android:layout_width="wrap_content"
105	android:layout_height="wrap_content"
106	android:text="@string/btn_web"
107	android:textSize="12sp" />
108	
109	<Button
110	android:id="@+id/btnDetail"
111	android:layout_width="wrap_content"
112	android:layout_height="wrap_content"
113	android:text="@string/btn_detail"
114	android:textSize="12sp" />
115	</LinearLayout>
116	</LinearLayout>
117	</LinearLayout>
118	</androidx.cardview.widget.CardView>

item_highlight.xml

Tabel 11. Source Code item_highlight.xml XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="250.dp"
6	android:layout_height="150.dp"
7	android:layout_marginEnd="12.dp"
8	app:cardCornerRadius="20.dp">
9	
10	<ImageView
11	android:id="@+id/ivHighlight"
12	android:layout_width="match_parent"
13	android:layout_height="match_parent"
14	android:scaleType="centerCrop" />
15	</androidx.cardview.widget.CardView>

fragment_list.xml

Tabel 12. Source Code fragment_list.xml XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
	xmlns:android="http://schemas.android.com/apk/res/andro
	id"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:orientation="vertical">
7	
8	<RelativeLayout
9	android:layout_width="match_parent"
10	android:layout_height="?attr/actionBarSize"
11	android:paddingHorizontal="16.dp"
12	android:background="?attr/colorPrimary">
13	
14	<TextView
15	android:layout_width="wrap_content"
16	android:layout_height="wrap_content"
17	android:layout_centerVertical="true"
18	android:text="@string/app_name"
19	android:textColor="@android:color/white"
20	android:textSize="20sp"
21	android:textStyle="bold"/>
22	
23	<ImageButton
24	android:id="@+id/btnLanguage"
25	android:layout_width="48dp"
26	android:layout_height="48dp"
27	android:layout_alignParentEnd="true"
28	android:layout_centerVertical="true"
29	
	android:background="?attr/selectableItemBackgroundBorde
	rlless"
30	
	android:contentDescription="@string/title_language"
31	android:src="@drawable/ic_language"
32	app:tint="@android:color/white" />
33	</RelativeLayout>
34	
35	<androidx.core.widget.NestedScrollView
36	android:layout_width="match_parent"
37	android:layout_height="match_parent">
38	
39	<LinearLayout
40	android:layout_width="match_parent"

```

41         android:layout_height="wrap_content"
42         android:orientation="vertical"
43         android:paddingBottom="16.dp">
44
45         <TextView
46             android:layout_width="wrap_content"
47             android:layout_height="wrap_content"
48             android:text="@string/title_highlight"
49             android:textSize="18sp"
50             android:textStyle="bold"
51             android:layout_margin="16.dp"/>
52
53         <androidx.recyclerview.widget.RecyclerView
54             android:id="@+id/rvHighlight"
55             android:layout_width="match_parent"
56             android:layout_height="wrap_content"
57             android:clipToPadding="false"
58             android:orientation="horizontal"
59             android:paddingHorizontal="16.dp"
60
61         app:layoutManager="androidx.recyclerview.widget.LinearL
62         ayoutManager" />
63
64         <TextView
65             android:layout_width="wrap_content"
66             android:layout_height="wrap_content"
67             android:text="@string/title_all_items"
68             android:textSize="18sp"
69             android:textStyle="bold"
70             android:layout_margin="16.dp"/>
71
72         <androidx.recyclerview.widget.RecyclerView
73             android:id="@+id/rvLego"
74             android:layout_width="match_parent"
75             android:layout_height="wrap_content"
76             android:nestedScrollingEnabled="false"
77
78         app:layoutManager="androidx.recyclerview.widget.LinearL
79         ayoutManager" />
80
81     </LinearLayout>
82 </androidx.core.widget.NestedScrollView>
83 </LinearLayout>

```

fragment_detail.xml

Tabel 13. Source Code fragment_detail.xml XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
	xmlns:android="http://schemas.android.com/apk/res/andro
	id"
3	android:layout_width="match_parent"
4	android:layout_height="match_parent"
5	android:orientation="vertical"
6	android:padding="16.dp">
7	
8	<ImageView
9	android:id="@+id/ivDetail"
10	android:layout_width="match_parent"
11	android:layout_height="300.dp"
12	android:scaleType="centerInside"/>
13	
14	<TextView
15	android:id="@+id/tvTitleDetail"
16	android:layout_width="wrap_content"
17	android:layout_height="wrap_content"
18	android:textSize="26sp"
19	android:textStyle="bold"
20	android:layout_marginTop="16.dp"/>
21	
22	<TextView
23	android:id="@+id/tvDescDetail"
24	android:layout_width="wrap_content"
25	android:layout_height="wrap_content"
26	android:textSize="16sp"
27	android:layout_marginTop="8.dp"/>
28	</LinearLayout>

fragment_language.xml

Tabel 14. Source Code fragment_language.xml XML Soal 1

1	<LinearLayout
	xmlns:android="http://schemas.android.com/apk/res/andro
	id"
2	android:layout_width="match_parent"
3	android:layout_height="match_parent"
4	android:gravity="center"
5	android:orientation="vertical"
6	android:padding="20.dp">
7	
8	<TextView

9	android:layout_width="wrap_content"
10	android:layout_height="wrap_content"
11	android:text="@string/title_language"
12	android:textSize="22sp"
13	android:layout_marginBottom="30.dp"/>
14	
15	<Button
16	android:id="@+id/btnIndo"
17	android:layout_width="match_parent"
18	android:layout_height="wrap_content"
19	android:text="Bahasa Indonesia" />
20	
21	<Button
22	android:id="@+id/btnEnglish"
23	android:layout_width="match_parent"
24	android:layout_height="wrap_content"
25	android:layout_marginTop="10.dp"
26	android:text="English" />
27	</LinearLayout>

nav_graph.xml

Tabel 15. Source Code nav_graph.xml XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<navigation
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:id="@+id/nav_graph"
5	app:startDestination="@id/listFragment">
6	
7	<fragment
8	android:id="@+id/listFragment"
9	
	android:name="com.example.modul3xml.ListFragment"
10	android:label="Daftar Lego">
11	
12	<action
13	android:id="@+id/action_list_to_detail"
14	app:destination="@id/detailFragment" />
15	
16	<action
17	
	android:id="@+id/action_listFragment_to_languageFragment"
18	app:destination="@id/languageFragment" />
19	</fragment>

20	
21	<fragment
22	android:id="@+id/detailFragment"
23	
	android:name="com.example.modul3xml.DetailFragment"
24	android:label="Detail Lego" />
25	
26	<fragment
27	android:id="@+id/languageFragment"
28	
	android:name="com.example.modul3xml.LanguageFragment"
29	android:label="Ganti Bahasa" />
30	
31	</navigation>

strings.xml (en)

Tabel 16. Source Code strings.xml (en) XML Soal 1

1	<resources>
2	<string name="app_name">Lego Collection</string>
3	<string name="btn_web">Web</string>
4	<string name="btn_detail">Detail</string>
5	<string
	name="title_highlight">Highlight
	Items</string>
6	<string
	name="title_all_items">All
	Collections</string>
7	<string name="label_year">Year:</string>
8	<string name="label_theme">Theme:</string>
9	<string
	name="title_language">Change
	Language</string>
10	</resources>

strings.xml (in)

Tabel 17. Source Code strings.xml (in) XML Soal 1

1	<resources>
2	<string name="app_name">Koleksi Lego</string>
3	<string name="btn_web">Situs</string>
4	<string name="btn_detail">Detail</string>
5	<string name="title_highlight">Item Sorotan</string>
6	<string
	name="title_all_items">Semua
	Koleksi</string>
7	<string name="label_year">Tahun:</string>
8	<string name="label_theme">Tema:</string>
9	<string name="title_language">Ganti Bahasa</string>
10	</resources>

2. Jetpack Compose

MainActivity.kt Compose

Tabel 18. Source Code MainActivity.kt Compose Soal 1

1	package com.example.modul3compose		
2			
3	import android.os.Bundle		
4	import androidx.activity.ComponentActivity		
5	import androidx.activity.compose.setContent		
6	import androidx.compose.foundation.layout.fillMaxSize		
7	import androidx.compose.material3.MaterialTheme		
8	import androidx.compose.material3.Surface		
9	import androidx.compose.ui.Modifier		
10	import androidx.navigation.compose.NavHost		
11	import androidx.navigation.compose.composable		
12	import		
	androidx.navigation.compose.rememberNavController		
13			
14	class MainActivity : ComponentActivity() {		
15	override fun onCreate(savedInstanceState: Bundle?) {		
16	super.onCreate(savedInstanceState)		
17			
18	val legoList = getDummyLegoList()		
19			
20	setContent {		
21	MaterialTheme {		
22	Surface(modifier	=	
	Modifier.fillMaxSize(),	=	
	MaterialTheme.colorScheme.background) {		
23			
24	val	navController	=
	rememberNavController()		
25			
26	NavHost(navController	=	
	navController, startDestination = "list_screen") {		
27			
28	composable("list_screen") {		
29	ListScreen(		
30	legoList = legoList,		
31	onNavigateToDetail = {		{
	id -> navController.navigate("detail_screen/\$id") },		
32	onNavigateToLanguage	=	
	{ navController.navigate("language_screen") }		
33	)		
34	}		
35			

36	composable("detail_screen/{legoId}") { backStackEntry -
37	> val id =
	backStackEntry.arguments?.getString("legoId")?.toIntOrNull()
38	val selectedLego =
	legoList.find { it.id == id }
39	
40	if (selectedLego != null) {
41	DetailScreen(lego =
	selectedLego)
42	}
43	}
44	
45	composable("language_screen") {
46	LanguageScreen()
47	}
48	}
49	}
50	}
51	}
52	}
53	}

Lego.kt Compose

Tabel 19. Source Code Lego.kt Compose Soal 1

1	package com.example.modul3compose
2	
3	data class Lego(
4	val id: Int,
5	val title: String,
6	val year: String,
7	val theme: String,
8	val description: String,
9	val imageRes: Int,
10	val webUrl: String
11	)
12	
13	fun getDummyLegoList(): List<Lego> {
14	return listOf(
15	Lego(1, "Millennium Falcon", "2017", "Star Wars",
	"Pesawat tempur legendaris Han Solo.", R.drawable.lego1,
	"https://www.lego.com"),

16	Lego(2, "Hogwarts Castle", "2018", "Harry Potter", "Kastil sekolah sihir Hogwarts.", R.drawable.lego2, "https://www.lego.com"),
17	Lego(3, "Taj Mahal", "2021", "Architecture", "Replika bangunan Taj Mahal di India.", R.drawable.lego3, "https://www.lego.com"),
18	Lego(4, "Porsche 911", "2021", "Icons", "Mobil sport klasik Porsche 911.", R.drawable.lego4, "https://www.lego.com"),
19	Lego(5, "Eiffel Tower", "2022", "Architecture", "Menara ikonik dari kota Paris.", R.drawable.lego5, "https://www.lego.com")
20	)
21	}

LegoItem.kt Compose

Tabel 20. Source Code LegoItem.kt Compose Soal 1

1	package com.example.modul3compose
2	
3	import androidx.compose.foundation.Image
4	import androidx.compose.foundation.layout.*
5	import
	androidx.compose.foundation.shape.RoundedCornerShape
6	import androidx.compose.material3.*
7	import androidx.compose.runtime.Composable
8	import androidx.compose.ui.Alignment
9	import androidx.compose.ui.Modifier
10	import androidx.compose.ui.draw.clip
11	import androidx.compose.ui.graphics.Color
12	import androidx.compose.ui.layout.ContentScale
13	import androidx.compose.ui.res.painterResource
14	import androidx.compose.ui.res.stringResource
15	import androidx.compose.ui.text.font.FontWeight
16	import androidx.compose.ui.text.style.TextOverflow
17	import androidx.compose.ui.unit.dp
18	import androidx.compose.ui.unit.sp
19	
20	@Composable
21	fun LegoItem(lego: Lego, onWebClick: () -> Unit, onDetailClick: () -> Unit) {
22	Card(
23	colors = CardDefaults.cardColors(containerColor = Color(0xFF2C2C2E)),
24	shape = RoundedCornerShape(16.dp),
25	modifier = Modifier.fillMaxWidth(),
26	elevation = CardDefaults.cardElevation(4.dp)

```

27     ) {
28         Row(modifier
29             Modifier.padding(16.dp).fillMaxWidth()) {
30             Image(
31                 painter = painterResource(id
32                     lego.imageRes),
33                 contentDescription = null,
34                 modifier = Modifier.size(100.dp,
35                     140.dp).clip(RoundedCornerShape(12.dp)),
36                 contentScale = ContentScale.Crop
37             )
38
39             Column(modifier = Modifier.padding(start =
40                 16.dp).weight(1f)) {
41                 // Baris 1: Judul & Tahun
42                 Row(modifier = Modifier.fillMaxWidth(),
43                     horizontalArrangement = Arrangement.SpaceBetween) {
44                     Text(text = lego.title, color =
45                         Color.White, fontSize = 18.sp, fontWeight =
46                         FontWeight.Bold, modifier = Modifier.weight(1f),
47                         maxLines = 1, overflow = TextOverflow.Ellipsis)
48                     Text(text = lego.year, color =
49                         Color.LightGray, fontSize = 14.sp)
50                 }
51
52                 Spacer(modifier = Modifier.height(8.dp))
53
54                 // Baris 2: Tema & Deskripsi
55                 Row {
56                     Text(text
57                         stringResource(R.string.label_theme), color
58                         Color.LightGray, fontSize = 13.sp, fontWeight
59                         FontWeight.Bold)
60                     Text(text = " ${lego.theme}", color
61                         = Color.LightGray, fontSize = 13.sp)
62                 }
63                 Text(text = lego.description, color =
64                     Color.Gray, fontSize = 12.sp, maxLines = 2, overflow =
65                     TextOverflow.Ellipsis, modifier = Modifier.padding(top =
66                     4.dp))
67
68                 Spacer(modifier
69                     Modifier.height(12.dp))
70
71                 // Baris 3: Tombol
72                 Row(modifier = Modifier.fillMaxWidth(),
73                     horizontalArrangement = Arrangement.End) {

```

56	TextButton(onClick = onWebClick) {
57	Text(stringResource(R.string.btn_web), color =
	Color(0xFF98A7F5))
58	}
59	Button(onClick = onDetailClick,
	colors = ButtonDefaults.buttonColors(containerColor =
	Color(0xFF98A7F5))) {
60	Text(stringResource(R.string.btn_detail), color =
	Color.Black)
61	}
62	}
63	}
64	}
65	}
66	}

AppScreens.kt Compose

Tabel 21. Source Code AppScreens.kt Compose Soal 1

1	package com.example.modul3compose
2	
3	import android.app.Activity
4	import android.content.Context
5	import android.content.Intent
6	import android.net.Uri
7	import androidx.compose.foundation.Image
8	import androidx.compose.foundation.layout.*
9	import androidx.compose.foundation.lazy.LazyColumn
10	import androidx.compose.foundation.lazy.LazyRow
11	import androidx.compose.foundation.lazy.items
12	import
	androidx.compose.foundation.shape.RoundedCornerShape
13	import androidx.compose.material.icons.Icons
14	import androidx.compose.material.icons.filled.Language
15	import androidx.compose.material3.*
16	import androidx.compose.runtime.Composable
17	import androidx.compose.ui.Alignment
18	import androidx.compose.ui.Modifier
19	import androidx.compose.ui.draw.clip
20	import androidx.compose.ui.layout.ContentScale
21	import androidx.compose.ui.platform.LocalContext
22	import androidx.compose.ui.res.painterResource
23	import androidx.compose.ui.res.stringResource
24	import androidx.compose.ui.text.font.FontWeight
25	import androidx.compose.ui.unit.dp

```

26 import androidx.compose.ui.unit.sp
27 import java.util.Locale
28
29 @OptIn(ExperimentalMaterial3Api::class)
30 @Composable
31 fun ListScreen(legoList: List<Lego>, onNavigateToDetail:
  (Int) -> Unit, onNavigateToLanguage: () -> Unit) {
32     val context = LocalContext.current
33     Scaffold(
34         topBar = {
35             TopAppBar(
36                 title = {
37                     Text(stringResource(R.string.app_name), fontWeight =
38                         FontWeight.Bold) },
39                 actions = {
40                     IconButton(onClick =
41                         onNavigateToLanguage) {
42                         Icon(Icons.Default.Language,
43                             contentDescription = "Lang") } }
44             )
45         }
46     ) { padding ->
47         LazyColumn(
48             modifier =
49                 Modifier.padding(padding).fillMaxSize(),
50                 contentPadding = PaddingValues(16.dp),
51                 verticalArrangement =
52                 Arrangement.spacedBy(16.dp)
53             ) {
54                 // highlight
55                 item {
56                     Text(stringResource(R.string.title_highlight),
57                         fontWeight = FontWeight.Bold, fontSize = 18.sp)
58                     LazyRow(modifier = Modifier.padding(top =
59                         8.dp), horizontalArrangement =
60                         Arrangement.spacedBy(12.dp)) {
61                         items(legoList) { lego ->
62                             Image(
63                                 painter =
64                                 painterResource(lego.imageRes), contentDescription =
65                                 null,
66                                 modifier =
67                                 Modifier.size(260.dp,
68                                     150.dp).clip(RoundedCornerShape(16.dp)),
69                                 contentScale =
70                                 ContentScale.Crop
71                             )
72                         }
73                     }
74                 }
75             }
76         }
77     }
78 }

```

```

57         }
58         Spacer(modifier =
Modifier.height(16.dp))
59
60         Text(stringResource(R.string.title_all_items),
fontWeight = FontWeight.Bold, fontSize = 18.sp)
61     }
62     // main list
63     items(legoList) { lego ->
64         LegoItem(
65             lego = lego,
66             onClick = {
context.startActivity(Intent(Intent.ACTION_VIEW,
Uri.parse(lego.webUrl))) },
67             onDetailClick = {
onNavigateToDetail(lego.id) }
68         )
69     }
70 }
71 }
72 }
73
74 @Composable
75 fun DetailScreen(lego: Lego) {
76     Column(modifier =
Modifier.fillMaxSize().padding(16.dp),
horizontalAlignment = Alignment.CenterHorizontally) {
77         Image(painter = painterResource(lego.imageRes),
contentDescription = null, modifier =
Modifier.size(200.dp).clip(RoundedCornerShape(16.dp)),
contentScale = ContentScale.Crop)
78         Spacer(modifier = Modifier.height(16.dp))
79         Text(text = lego.title, fontSize = 24.sp,
fontWeight = FontWeight.Bold)
80         Text(text =
"${stringResource(R.string.label_year)} ${lego.year}",
fontSize = 16.sp)
81         Text(text =
"${stringResource(R.string.label_theme)} ${lego.theme}",
fontSize = 16.sp)
82         Spacer(modifier = Modifier.height(16.dp))
83         Text(text = lego.description, fontSize = 14.sp)
84     }
85 }
86
87 @Composable

```



```

88 fun LanguageScreen() {
89     val context = LocalContext.current
90     Column(modifier = Modifier.fillMaxSize(),
verticalArrangement = Arrangement.Center,
horizontalAlignment = Alignment.CenterHorizontally) {
91         Text(stringResource(R.string.title_language),
fontSize = 22.sp, fontWeight = FontWeight.Bold)
92         Spacer(modifier = Modifier.height(24.dp))
93         Button(onClick = { updateLocale(context, "in") })
{ Text("Bahasa Indonesia") }
94         Spacer(modifier = Modifier.height(8.dp))
95         Button(onClick = { updateLocale(context, "en") })
{ Text("English") }
96     }
97 }
98
99 fun updateLocale(context: Context, lang: String) {
100     val locale = Locale(lang)
101     Locale.setDefault(locale)
102     val config = context.resources.configuration
103     config.setLocale(locale)
104     context.resources.updateConfiguration(config,
context.resources.displayMetrics)
105     (context as? Activity)?.recreate() // Refresh
activity
106 }

```

strings.xml (in)

Tabel 22. Source Code strings.xml (in)Compose Soal 1

```

1 <resources>
2     <string name="app_name">Koleksi Lego</string>
3     <string name="btn_web">Situs</string>
4     <string name="btn_detail">Detail</string>
5     <string name="title_highlight">Item Sorotan</string>
6     <string
name="title_all_items">Semua
Koleksi</string>
7     <string name="label_year">Tahun:</string>
8     <string name="label_theme">Tema:</string>
9     <string name="title_language">Ganti Bahasa</string>
10 </resources>

```

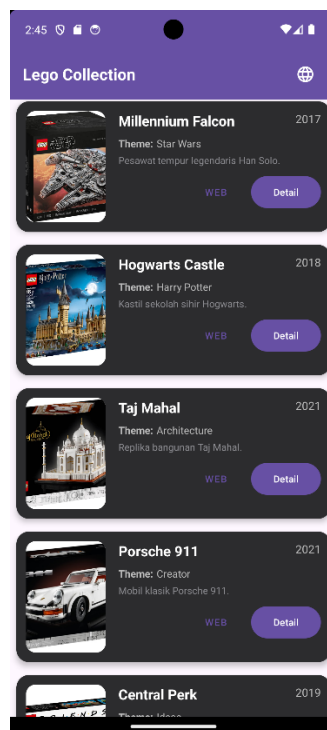
strings.xml (en)

Tabel 23. Source Code strings.xml (en) Compose Soal 1

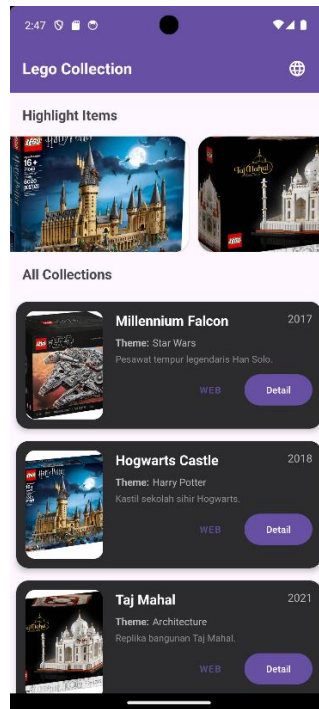
1	<resources>
2	<string name="app_name">Lego Collection</string>
3	<string name="btn_web">Web</string>
4	<string name="btn_detail">Detail</string>
5	<string name="title_highlight">Highlight
	Items</string>
6	<string name="title_all_items">All
	Collections</string>
7	<string name="label_year">Year:</string>
8	<string name="label_theme">Theme:</string>
9	<string name="title_language">Change
	Language</string>
10	</resources>

B. Output Program

1. XML



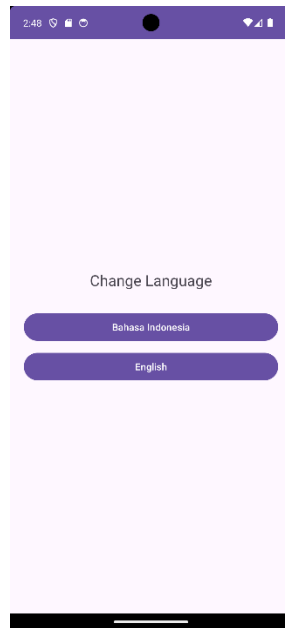
Gambar 4. Screenshot Hasil Jawaban Soal 1 XML



Gambar 5. Screenshot Hasil Jawaban Soal 1 XML

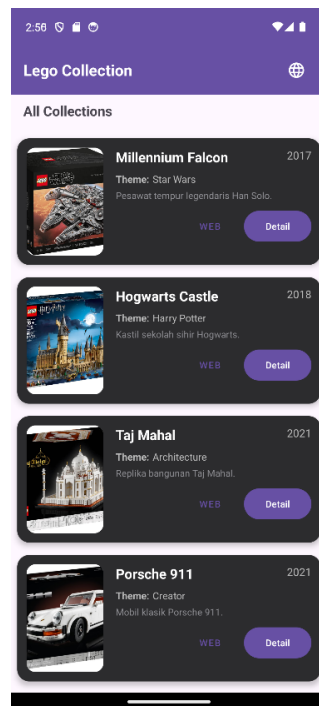


Gambar 6. Screenshot Hasil Jawaban Soal 1 XML

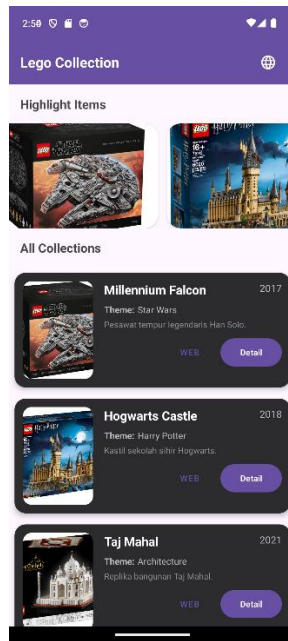


Gambar 7. Screenshot Hasil Jawaban Soal 1 XML

2. Jetpack Compose



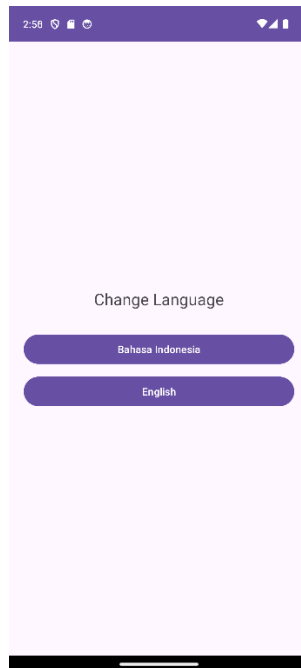
Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 9. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 10. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 11. Screenshot Hasil Jawaban Soal 1 Compose

C. Pembahasan

1. XML

MainActivity.kt

Pada file 'MainActivity.kt', class 'MainActivity' bertindak sebagai komponen utama yang mewarisi sifat dari 'AppCompatActivity'. Di dalam class ini, dideklarasikan sebuah variabel bernama 'binding' dengan tipe 'ActivityMainBinding' menggunakan kata kunci 'lateinit var'. Penggunaan 'ViewBinding' ini bertujuan untuk menggantikan metode 'findViewById', sehingga interaksi dengan komponen antarmuka menjadi lebih aman dan ringkas. Alur program dimulai pada fungsi 'onCreate()', yang merupakan titik masuk utama saat aktivitas dibuat oleh sistem Android. Di dalam fungsi 'onCreate()', program memanggil fungsi 'ActivityMainBinding.inflate(layoutInflater)' untuk mengubah file desain XML menjadi objek pendukung yang dapat dikenali oleh kode Kotlin. Setelah proses inflasi tersebut selesai, hasilnya disimpan ke dalam variabel 'binding'. Langkah terakhir adalah memanggil fungsi 'setContentView(binding.root)', yang berfungsi untuk menampilkan seluruh hierarki tampilan pada layar perangkat dengan mengambil elemen 'root' dari hasil binding tersebut. Karena aplikasi ini menggunakan arsitektur 'Single Activity', maka

'MainActivity' ini murni berfungsi sebagai wadah atau 'host' bagi fragment-fragment lain yang akan dimuat melalui 'Navigation Component'.

Lego.kt

Pada file 'Lego.kt', didefinisikan sebuah 'data class' bernama 'Lego' yang berfungsi sebagai model data untuk menampung informasi item seperti 'id', 'title', 'year', 'theme', 'pieces', 'description', 'imageRes', dan 'webUrl'. Agar objek dari 'class' ini dapat dikirim antar fragment melalui 'Navigation Component', 'class' ini mengimplementasikan antarmuka 'Parcelable'. Di dalam 'class' tersebut, terdapat fungsi 'writeToParcel()' yang bertugas menuliskan seluruh properti data ke dalam objek 'Parcel'. Sebaliknya, terdapat 'constructor' sekunder yang berfungsi membaca kembali data dari 'Parcel' untuk merekonstruksi objek 'Lego'. Terakhir, bagian 'companion object CREATOR' diimplementasikan untuk memenuhi standar 'Parcelable', di mana fungsi 'createFromParcel()' digunakan untuk menciptakan instance baru dari data yang telah dikemas, dan fungsi 'newArray()' untuk membuat array dari objek tersebut.

LegoViewModel.kt

Pada file 'LegoViewModel.kt', terdapat sebuah 'class' bernama 'LegoViewModel' yang merupakan turunan dari 'ViewModel'. 'class' ini berfungsi sebagai penyedia data untuk antarmuka pengguna, di mana data yang disimpan di dalamnya akan tetap terjaga meskipun terjadi perubahan konfigurasi pada perangkat seperti rotasi layar. Di dalam 'class' ini, dideklarasikan sebuah variabel 'legoList' yang menampung daftar data menggunakan fungsi 'listOf()'. Variabel tersebut berisi kumpulan objek dari 'class' 'Lego' yang menyertakan berbagai informasi seperti 'id', judul, tahun, tema, jumlah kepingan, deskripsi, referensi gambar dari 'R.drawable', serta tautan situs web. Daftar data inilah yang nantinya akan dipanggil oleh 'ListFragment' melalui 'ViewModelProvider' untuk ditampilkan ke dalam 'RecyclerView'.

LegoAdapter.kt

Pada file 'LegoAdapter.kt', terdapat 'class' 'LegoAdapter' yang mewarisi 'RecyclerView.Adapter' untuk mengelola dan menampilkan daftar data ke dalam

'RecyclerView'. Di dalamnya, didefinisikan 'class' 'LegoViewHolder' yang menggunakan 'ItemLegoBinding' untuk memegang referensi seluruh komponen UI pada setiap baris item. Fungsi 'onCreateViewHolder()' berperan untuk menyiapkan tampilan dengan melakukan proses 'inflate' pada layout item, sementara fungsi 'getItemCount()' mengembalikan jumlah total data yang akan ditampilkan dari variabel 'listData'.

Logika utama pengisian data dilakukan di dalam fungsi 'onBindViewHolder()'. Di sini, program mengambil objek dari 'listData' sesuai urutan 'position' dan memetakan datanya ke komponen seperti 'tvTitle', 'tvYear', 'tvTheme', 'tvDescription', serta memasang gambar melalui 'ivLego'. Selain itu, fungsi ini menangani dua jenis interaksi; tombol 'btnWeb' menjalankan 'Intent' untuk membuka tautan situs web menggunakan 'Uri.parse()', sedangkan tombol 'btnDetail' berfungsi untuk memicu navigasi ke fragment detail dengan membawa data objek 'Lego' yang telah dibungkus ke dalam 'Bundle' melalui perintah 'navigate()'.

ListFragment.kt

Pada file 'ListFragment.kt', terdapat 'class' 'ListFragment' yang mewarisi 'Fragment' dan bertugas mengelola tampilan daftar utama. Program menggunakan variabel '_binding' dan 'binding' untuk menerapkan 'ViewBinding' agar dapat mengakses elemen UI pada 'fragment_list.xml'. Proses inisialisasi tata letak dilakukan di dalam fungsi 'onCreateView()' melalui perintah 'FragmentManager.inflate()', sementara fungsi 'onDestroyView()' bertugas membersihkan referensi binding untuk mencegah kebocoran memori.

Logika tampilan diatur di dalam fungsi 'onViewCreated()'. Program memulai dengan menginisialisasi variabel 'viewModel' melalui 'ViewModelProvider' untuk mengambil data daftar Lego. Selanjutnya, program mengatur dua jenis daftar: 'rvHighlight' dikonfigurasi dengan 'LinearLayoutManager' secara horizontal menggunakan 'HighlightAdapter', sedangkan 'rvLego' dikonfigurasi secara vertikal menggunakan 'LegoAdapter'. Terakhir, tombol 'btnLanguage' dipasang 'setOnClickListener' yang memicu fungsi 'findNavController().navigate()' untuk berpindah ke halaman pengaturan bahasa melalui 'Navigation Component'.

DetailFragment.kt

Pada file 'DetailFragment.kt', terdapat 'class' 'DetailFragment' yang berfungsi untuk menampilkan informasi rinci dari sebuah item Lego yang dipilih. Sama seperti fragment lainnya, 'class' ini menggunakan teknik 'ViewBinding' dengan variabel '_binding' untuk menghubungkan kode Kotlin dengan tata letak 'fragment_detail.xml'. Proses inisialisasi antarmuka dilakukan pada fungsi 'onCreateView()', sementara fungsi 'onDestroyView()' digunakan untuk menghapus referensi binding guna menjaga efisiensi memori perangkat.

Logika pemrosesan data terjadi di dalam fungsi 'onViewCreated()'. Pertama, program mengambil data yang dikirimkan oleh fragment sebelumnya melalui fungsi 'arguments?.getParcelable()' dengan kunci 'data_lego'. Jika data tersebut berhasil diterima, program akan memperbarui komponen UI dengan memasang gambar pada 'ivDetail' dan judul pada 'tvTitleDetail'. Selanjutnya, program menggabungkan informasi tahun, tema, jumlah kepingan, dan deskripsi ke dalam satu variabel string bernama 'teksDetail' untuk ditampilkan secara sekaligus melalui komponen 'tvDescDetail'.

LanguageFragment.kt

Pada file 'LanguageFragment.kt', terdapat 'class' 'LanguageFragment' yang bertugas mengelola fitur pergantian bahasa dalam aplikasi. 'class' ini menggunakan 'ViewBinding' melalui variabel '_binding' untuk mengakses komponen pada layout 'fragment_language.xml'. Di dalam fungsi 'onViewCreated()', program melakukan inisialisasi 'binding' dan memasang 'setOnClickListener' pada tombol 'btnIndo' serta 'btnEnglish' yang masing-masing akan memanggil fungsi 'setLocale()' dengan kode bahasa yang sesuai. Fungsi utama 'setLocale()' bekerja dengan menciptakan objek 'Locale' baru berdasarkan kode bahasa yang diterima, lalu menetapkannya sebagai bahasa default melalui 'Locale.setDefault()'. Selanjutnya, program memperbarui konfigurasi perangkat menggunakan objek 'Configuration' dan memanggil fungsi 'updateConfiguration()' agar sumber daya aplikasi menyesuaikan dengan bahasa pilihan pengguna. Terakhir, perintah 'requireActivity().recreate()' dijalankan untuk memuat ulang seluruh 'MainActivity' sehingga perubahan bahasa langsung diterapkan di semua bagian antarmuka, sementara fungsi 'onDestroyView()' memastikan referensi 'binding' dibersihkan saat fragment dihancurkan.

HighlightAdapter.kt

Pada file 'HighlightAdapter.kt', didefinisikan sebuah 'class' bernama 'HighlightAdapter' yang berfungsi untuk mengelola tampilan item pada bagian sorotan atau carousel secara horizontal. 'class' ini menggunakan 'ItemHighlightBinding' untuk menghubungkan kode dengan layout 'item_highlight.xml'. Di dalamnya, terdapat 'class' 'ViewHolder' yang bertugas sebagai wadah untuk menyimpan referensi komponen antarmuka agar proses pemuatan data menjadi lebih efisien.

Fungsi 'onCreateViewHolder()' dipanggil untuk membuat instance baru dari 'ViewHolder' dengan melakukan proses 'inflate' pada layout item sorotan. Sementara itu, fungsi 'onBindViewHolder()' bertanggung jawab untuk menghubungkan data dari variabel 'list' ke komponen visual; dalam hal ini, program mengambil referensi gambar dari objek 'Lego' berdasarkan posisi tertentu dan memasangnya pada komponen 'ivHighlight'. Terakhir, fungsi 'getItemCount()' digunakan untuk memberitahu 'RecyclerView' mengenai jumlah total item yang tersedia di dalam daftar sorotan tersebut.

activity_main.xml

Pada file desain ini, digunakan komponen 'FragmentContainerView' yang berfungsi sebagai wadah utama untuk menampilkan konten aplikasi secara dinamis. Komponen ini memiliki 'android:id' berupa 'nav_host_fragment' dan mendefinisikan 'androidx.navigation.fragment.NavHostFragment' pada atribut 'android:name'. Pengaturan ini menjadikan area tersebut sebagai pusat kendali navigasi, di mana fragment-fragment akan bergantian muncul sesuai dengan alur yang dijalankan pengguna. Atribut 'app:navGraph' yang diarahkan ke '@navigation/nav_graph' berfungsi untuk menghubungkan wadah ini dengan file konfigurasi navigasi yang menentukan rute antar halaman. Selain itu, atribut 'app:defaultNavHost' disetel ke nilai 'true', yang memastikan bahwa tombol kembali (back button) pada perangkat sistem operasi Android akan terintegrasi secara otomatis dengan antrean navigasi di dalam fragment tersebut. Dengan struktur ini, 'MainActivity' tidak perlu lagi mengganti fragment secara manual melalui kode program, melainkan cukup melalui sistem 'Navigation Component'.

item_lego.xml

Pada file desain 'item_lego.xml', komponen 'CardView' digunakan sebagai kontainer utama untuk memberikan tampilan kartu dengan sudut melengkung 'app:cardCornerRadius' dan efek bayangan melalui 'app:cardElevation'. Di dalamnya, terdapat 'LinearLayout' dengan orientasi horizontal yang membagi tampilan menjadi dua bagian utama. Bagian kiri berisi 'CardView' tambahan yang membungkus 'ImageView' dengan ID 'ivLego' untuk menampilkan foto Lego secara proporsional menggunakan atribut 'scaleType="centerCrop"'. Bagian kanan kartu diisi oleh 'LinearLayout' vertikal yang menampung informasi detail. Program menggunakan 'RelativeLayout' untuk menempatkan judul 'tvTitle' di sisi kiri dan tahun 'tvYear' di sisi kanan secara sejajar. Di bawahnya, terdapat susunan teks untuk menampilkan label tema serta deskripsi singkat 'tvDescription' yang dibatasi jumlah barisnya menggunakan atribut 'maxLines'. Terakhir, terdapat bagian tombol di posisi bawah yang menggunakan 'gravity="end"', di mana 'btnWeb' berfungsi sebagai tombol teks dan 'btnDetail' berfungsi sebagai tombol utama untuk mengarahkan pengguna ke halaman rincian.

item_highlight.xml

Pada file desain 'item_highlight.xml', digunakan komponen 'CardView' sebagai elemen pembungkus utama yang dirancang khusus untuk tampilan carousel atau daftar horizontal. Komponen ini memiliki ukuran lebar tetap sebesar '250.dp' dan tinggi '150.dp' guna memastikan konsistensi visual saat item berjejer. Penggunaan atribut 'app:cardCornerRadius' sebesar '20.dp' memberikan tampilan sudut yang sangat melengkung agar terlihat lebih modern dan menonjol dibandingkan kartu daftar biasa. Di dalam kartu tersebut, terdapat satu komponen 'ImageView' dengan ID 'ivHighlight' yang mengisi seluruh ruang kartu melalui atribut 'match_parent'. Foto Lego yang dimuat ke dalam komponen ini diatur menggunakan 'android:scaleType="centerCrop"' agar gambar memenuhi bingkai tanpa merusak rasio aslinya. Desain yang ringkas ini difokuskan sepenuhnya pada aspek visual untuk menarik perhatian pengguna saat melihat bagian sorotan koleksi pada layar utama aplikasi.

fragment_list.xml

Pada file desain ini, digunakan 'LinearLayout' vertikal sebagai kontainer utama yang diawali dengan 'RelativeLayout' sebagai bar navigasi atas (top bar). Di dalamnya terdapat judul aplikasi dan 'ImageButton' dengan ID 'btnLanguage' untuk akses pengaturan bahasa. Bagian konten menggunakan 'NestedScrollView' agar seluruh halaman dapat digulir dengan lancar. Di dalamnya, terdapat dua 'RecyclerView' utama: 'rvHighlight' yang diatur secara horizontal untuk menampilkan item sorotan, dan 'rvLego' yang diatur secara vertikal untuk menampilkan seluruh koleksi. Penggunaan 'nestedScrollingEnabled="false"' pada 'rvLego' memastikan proses pengguliran ditangani sepenuhnya oleh kontainer utama agar tidak terjadi bentrokan navigasi.

fragment_detail.xml

Pada file desain ini, digunakan 'LinearLayout' dengan orientasi vertikal sebagai struktur utamanya. Bagian atas didominasi oleh 'ImageView' dengan ID 'ivDetail' yang memiliki tinggi '300.dp' dan menggunakan 'scaleType="centerInside"' untuk menampilkan gambar produk secara utuh tanpa terpotong. Di bawah gambar, terdapat 'TextView' dengan ID 'tvTitleDetail' yang menggunakan ukuran teks besar '26sp' dan gaya cetak tebal untuk menonjolkan nama item. Terakhir, komponen 'tvDescDetail' berfungsi sebagai penampung seluruh teks rincian informasi, mulai dari tahun rilis hingga deskripsi lengkap, dengan jarak '8.dp' dari judul agar tata letak tetap rapi dan mudah dibaca.

fragment_language.xml

Pada file desain ini, digunakan 'LinearLayout' vertikal sebagai kontainer utama dengan atribut 'gravity="center"' agar seluruh elemen berada tepat di tengah layar. Bagian atas diisi oleh 'TextView' yang menampilkan judul pengaturan bahasa dengan ukuran teks '22sp' dan jarak bawah '30.dp'. Di bawahnya, terdapat dua tombol utama, yaitu 'btnIndo' untuk memilih Bahasa Indonesia dan 'btnEnglish' untuk memilih Bahasa Inggris, yang keduanya diatur dengan lebar 'match_parent' agar mudah dijangkau oleh pengguna.

nav_graph.xml

Pada file 'nav_graph.xml', digunakan tag 'navigation' sebagai akar utama untuk mendefinisikan seluruh alur perpindahan antar layar dalam aplikasi. Atribut 'app:startDestination' diarahkan ke '@id/listFragment', yang berarti 'ListFragment' akan menjadi halaman pertama yang muncul saat aplikasi dijalankan. File ini berfungsi sebagai peta navigasi yang menyatukan berbagai 'fragment' ke dalam satu sistem koordinasi yang terpusat. Di dalam graf ini, terdapat tiga komponen 'fragment' utama, yaitu 'listFragment', 'detailFragment', dan 'languageFragment', yang masing-masing dihubungkan dengan class Kotlin terkait melalui atribut 'android:name'. Khusus pada 'listFragment', didefinisikan dua buah 'action' sebagai jalur perpindahan: 'action_list_to_detail' digunakan untuk berpindah ke layar rincian, sementara 'action_listFragment_to_languageFragment' digunakan untuk menuju ke pengaturan bahasa. Dengan adanya deklarasi 'action' ini, proses navigasi di dalam kode Kotlin dapat dipanggil secara aman menggunakan ID yang telah ditentukan.

strings.xml (en) & strings.xml (in)

Kedua file strings.xml ini berfungsi sebagai penyimpan sumber daya teks yang memungkinkan aplikasi memiliki fitur multibahasa atau lokalisasi. File pertama menyimpan teks dalam Bahasa Inggris sebagai bahasa default, sedangkan file kedua menyediakan terjemahannya dalam Bahasa Indonesia. Penggunaan tag <resources> memastikan bahwa semua label antarmuka, seperti 'app_name', 'btn_web', dan 'title_highlight', terpusat di satu lokasi sehingga memudahkan pengelolaan konten tanpa mengubah struktur kode program. Pemisahan ini sangat krusial agar aplikasi dapat beradaptasi secara otomatis saat pengguna memicu fungsi 'setLocale' pada pengaturan bahasa. Ketika fungsi 'setLocale' dijalankan dengan kode bahasa "in" untuk Indonesia atau "en" untuk Inggris, sistem Android akan mencari nilai yang sesuai dari file-file ini dan memperbarui teks pada komponen seperti 'tvTitleDetail' atau 'btnDetail' secara instan. Dengan pendekatan ini, pengembang dapat menjaga kode tetap bersih dan mempermudah proses penerjemahan aplikasi ke berbagai bahasa lainnya di masa mendatang.

2. Jetpack Compose

MainActivity.kt

Pada file MainActivity.kt, kelas 'MainActivity' berfungsi sebagai titik masuk utama aplikasi yang menggunakan Jetpack Compose dengan mewarisi 'ComponentActivity'. Di dalam metode 'onCreate', fungsi 'setContent' digunakan untuk mendefinisikan seluruh struktur antarmuka pengguna, di mana fungsi 'MaterialTheme' dan fungsi 'Surface' bertindak sebagai pembungkus untuk menerapkan tema dan latar belakang dasar. Navigasi antar layar dikelola oleh fungsi 'rememberNavController' yang menyimpan status perpindahan halaman, yang kemudian diintegrasikan ke dalam fungsi 'NavHost'. Fungsi 'NavHost' berperan sebagai pusat kendali rute dengan menetapkan "list_screen" sebagai 'startDestination'. Melalui fungsi 'composable', aplikasi mendaftarkan tiga rute utama: pertama adalah "list_screen" yang memanggil fungsi 'ListScreen' dengan membawa data dari 'getDummyLegoList'. Kedua adalah rute dinamis "detail_screen/{legoId}" yang menggunakan 'backStackEntry' untuk mengambil parameter melalui fungsi 'getString', mengonversinya dengan fungsi 'toIntOrNull', lalu mencari data yang sesuai menggunakan fungsi 'find' untuk ditampilkan dalam fungsi 'DetailScreen'. Terakhir, rute "language_screen" didaftarkan untuk menampilkan fungsi 'LanguageScreen' yang menangani pengaturan bahasa aplikasi.

Lego.kt

Pada bagian ini, 'data class' 'Lego' berperan sebagai cetak biru atau model data utama yang menyimpan seluruh informasi penting mengenai koleksi mainan tersebut. Struktur ini memastikan bahwa setiap objek memiliki atribut yang lengkap, mulai dari identitas unik hingga rujukan sumber daya gambar. Penggunaan model data yang terpusat seperti ini sangat membantu agar aliran data di dalam komponen Compose tetap teratur, sehingga fungsi-fungsi lain dapat mengakses properti seperti 'title' atau 'theme' dengan cara yang aman dan konsisten.

Untuk mengisi aplikasi dengan konten, disediakan fungsi 'getDummyLegoList' yang menghasilkan sekumpulan data simulasi melalui pemanggilan fungsi 'listOf'. Di dalam badan fungsi tersebut, beberapa objek 'Lego' diinisialisasi secara manual dengan detail spesifik seperti "Millennium Falcon" atau "Hogwarts Castle". Data yang dikembalikan oleh fungsi

'getDummyLegoList' inilah yang nantinya akan diproses oleh fungsi 'MainActivity' untuk kemudian dipetakan ke dalam antarmuka pengguna melalui daftar komponen yang dinamis.

LegoItem.kt

Pada file ini, fungsi 'LegoItem' menggunakan fungsi 'Card' sebagai kontainer utama dengan sudut melengkung 'RoundedCornerShape'. Tata letaknya disusun menggunakan fungsi 'Row' untuk memisahkan area visual dan informasi teks secara horizontal. Bagian kiri diisi oleh fungsi 'Image' yang menampilkan gambar produk dengan potongan rapi melalui fungsi 'clip' agar selaras dengan desain kartu yang modern. Di sisi kanan, fungsi 'Column' digunakan untuk menata informasi secara vertikal, mulai dari judul hingga deskripsi dengan fungsi 'Text'. Penggunaan fungsi 'stringResource' pada label statis memastikan teks tersebut tetap mendukung fitur lokalisasi bahasa. Sebagai penutup, terdapat fungsi 'Row' di bagian bawah yang menampung fungsi 'TextButton' dan fungsi 'Button' sebagai elemen interaktif untuk mengarahkan pengguna ke situs web atau layar rincian koleksi.

AppScreens.kt

Pada komponen utama, fungsi 'ListScreen' bertindak sebagai halaman daftar koleksi yang disusun menggunakan fungsi 'Scaffold' untuk menyediakan struktur 'TopAppBar' di bagian atas. Di dalam bilah navigasi tersebut, terdapat fungsi 'IconButton' dengan ikon bahasa yang akan memicu fungsi 'onNavigateToLanguage' ketika diklik oleh pengguna. Konten utama layar ini dikelola oleh fungsi 'LazyColumn' untuk menampilkan daftar secara vertikal, yang di dalamnya juga terdapat fungsi 'LazyRow' guna menyusun item sorotan (highlight) dalam arah horizontal. Saat pengguna ingin melihat situs web resmi, fungsi 'LocalContext.current' membantu menyediakan konteks agar aplikasi dapat menjalankan fungsi 'startActivity' untuk membuka tautan melalui 'Intent' eksternal.

Ketika pengguna memilih salah satu item, fungsi 'DetailScreen' akan mengambil alih untuk menampilkan informasi lengkap dalam tata letak fungsi 'Column' yang diposisikan di tengah layar. Foto produk ditampilkan dalam ukuran besar melalui fungsi 'Image' dengan sudut melengkung hasil dari fungsi 'clip', sementara informasi pendukung seperti judul dan deskripsi disajikan menggunakan fungsi 'Text'. Sementara itu, untuk pengaturan bahasa, fungsi 'LanguageScreen' menyediakan antarmuka sederhana dengan dua fungsi 'Button' yang

masing-masing akan memanggil fungsi 'updateLocale'. Fungsi 'updateLocale' bekerja secara teknis dengan mengubah konfigurasi bahasa melalui fungsi 'setLocale' dan fungsi 'updateConfiguration', lalu diakhiri dengan pemanggilan fungsi 'recreate' agar seluruh tampilan aplikasi segera diperbarui sesuai dengan bahasa yang dipilih.

Strings.xml (in) & strings.xml (en)

Kedua file strings.xml ini berfungsi sebagai penyedia sumber daya teks yang memungkinkan aplikasi mendukung fitur lokalisasi antara Bahasa Indonesia dan Bahasa Inggris. Dengan menyimpan semua label seperti 'app_name', 'btn_web', dan 'title_language' di dalam tag <resources>, pengembang dapat memisahkan konten teks dari logika program sehingga lebih mudah dikelola. Sistem Android akan secara otomatis mengambil teks dari file yang sesuai ketika fungsi 'updateLocale' dijalankan atau saat pengguna mengubah preferensi bahasa pada perangkat, sehingga seluruh komponen antarmuka dapat diperbarui secara instan tanpa mengubah struktur kode.

D. Tautan Git

<https://github.com/rikafaulianarahmi/MODUL-MOBILE/tree/main/MODUL%203>

2. RecyclerView masih digunakan karena:

- RecyclerView adalah bagian dari ekosistem Android View berbasis XML yang sudah sangat matang. Banyak project lama dan library pihak ketiga masih menggunakan sistem ini. Selama masih ada project yang menggunakan XML layout, RecyclerView tetap relevan.
- RecyclerView memberikan control yang lebih tinggi terhadap proses recycling view. Developer dapat mengimplementasikan:
 - RecyclerViewPool untuk berbagi pool view antar RecyclerView
 - ItemAnimator kustom untuk animasi item
 - ItemDecoration untuk separator dan dekorasi kustom
 - Multiple ViewHolder type secara eksplisit

- Untuk list yang sangat besar dan kompleks dengan banyak tipe item, RecyclerView memberikan optimisasi yang bisa dikontrol langsung. DiffUtil pada RecyclerView juga sangat powerful untuk update partial yang efisien.
- Banyak library populer seperti ItemTouchHelper (drag & drop, swipe), SnapHelper (untuk carousel), dan berbagai library third-party masih lebih optimal digunakan bersama RecyclerView.
- RecyclerView sudah ada sejak 2014 dan memiliki dokumentasi, tutorial, serta Stack Overflow answer yang sangat lengkap. Ini penting untuk pemula yang sedang belajar.

Jadi, LazyColumn memang lebih singkat dan modern untuk Jetpack Compose, tetapi RecyclerView tetap digunakan di project berbasis XML karena kompatibilitas, fleksibilitas, dan ekosistem yang sudah terbukti matang.

3. Alasan untuk menerapkan Clean Architecture:

- Setiap layer memiliki tanggung jawab yang jelas dan tidak saling campur. Misalnya, UI hanya mengurus tampilan, bukan mengambil data dari API. Ini membuat kode lebih mudah dipahami dan dikelola.
- Karena setiap layer terpisah dan bergantung pada abstraksi (interface), kita bisa melakukan unit test pada setiap layer secara independen. Business logic di Domain Layer bisa dites tanpa perlu Android framework.
- Ketika ada bug atau perubahan requirement, developer langsung tahu harus mengubah di layer mana. Jika API berubah, hanya Data Layer yang perlu diubah tanpa menyentuh UI atau Business Logic.
- Saat project berkembang dan tim bertambah, Clean Architecture memudahkan pembagian tugas. Satu developer bisa mengerjakan UI sementara developer lain mengerjakan Data Layer secara paralel.
- Business logic tidak bergantung pada Android framework, sehingga bisa dipindahkan ke platform lain (seperti backend Kotlin) dengan perubahan minimal.
- Tanpa arsitektur yang jelas, kode cenderung menumpuk di satu tempat (misalnya semua logic di Activity/Fragment). Clean Architecture mencegah hal ini.